

T-ACL: A Closure-Status Calculus for AI Governance Decisions

Abstract

AI governance has become rich in assurance materials: model cards, dataset documentation, bias audits, risk assessments, legal memoranda, and contestability records. These materials help reviewers see what has been checked, but they leave a narrower closure question under-specified: when is a deployment claim ready for acceptance, ready for rejection, or still open because a required dependency has not been resolved?

T-ACL is a finite closure-status interface for that decision point. It evaluates compound governance claims over three statuses: T permits closure as acceptance for a stated institutional purpose; F permits closure as rejection; C marks corrective non-closure. The central contribution is closure-control separation. Remediable non-closure, closure-level rejection, and non-waivable blocking failure receive different formal treatments, so a negative finding cannot be used interchangeably as a repairable concern, an ordinary rejection, or a hard legal/safety stop. The interface separates factual conjunction $\wedge_f$ from deliberative conjunction $\wedge_d$, uses ValidStop_t and recursive Block_t to propagate validated hard stops, and requires C to carry reason, owner, expiry, next-action, and appeal fields.

The article develops this interface through a small status algebra, a pair-labelled finite-stage evaluation procedure, and three public-record reconstructions: EEOC v. iTutorGroup, Michigan’s MiDAS unemployment-fraud system, and the Dutch SyRI judgment. The formal propositions are supporting design guarantees: deterministic evaluation at a fixed stage, preservation of the ordinary T/F fragment, procedural conflict consolidation, and finite computability once typed atomic statuses are supplied. The cases are representational adequacy tests: they ask whether the calculus can encode public-record insufficiency, alleged rule-level exclusion, due-process failure, rights-based legal blocking, audit incompleteness, and corrected-system review without collapsing them into a binary approval result.

Keywords

AI governance; closure-status calculus; typed atomic status generation; algorithmic accountability; AI assurance; formal governance; legal blocking conditions; procedural non-closure

1 Introduction

Public agencies and private organizations often review AI systems under conditions that are neither ready for approval nor ready for final rejection. A hiring model may pass an aggregate performance threshold while leaving subgroup impact unresolved. A welfare-risk classifier may have administrative utility while weakening contestability for affected people. A policy model may support a public intervention while remaining sensitive to drift, data provenance, or legal authority. In such cases, the practical mistake is not simply choosing the wrong truth value. It is closing the review too early, or leaving it open without a responsible path to resolution.

Recent controversies make the closure problem concrete. The Dutch childcare-benefits scandal showed how automated risk classification can become publicly harmful when review, contestability, and institutional accountability fail (Amnesty International 2021). New York City’s Local Law 144 on automated employment decision tools raises a related but narrower issue: even when bias audits and public notices are required, a reviewing institution still needs a defensible status for a deployment whose performance, fairness, legal, and audit dependencies do not all settle at once (New York City Department of Consumer and Worker Protection 2023). These examples motivate the modelling problem; the documentary evaluation later in the article uses public records from three domains: employment screening, unemployment-benefit fraud adjudication, and public-sector risk profiling.

The research question is:

Can a finite closure-status calculus distinguish closure as acceptance, closure as rejection, corrective non-closure, and non-waivable blocking failure in an auditable AI governance workflow?

The article makes four substantive contributions, ordered by their role in the argument. First, it introduces closure-control separation: remediable non-closure, closure-level rejection, and non-waivable blocking failure are distinct closure-control paths. Second, it defines a minimum reason-coded account of corrective non-closure, in which C is backed by reason, owner, expiry, next action, and appeal metadata. Third, it formalizes valid hard stops through ValidStop_t and recursive Block_t , with role separation and contestability as institutional preconditions. Fourth, it treats typed atomic status generation as an interface protocol, so heterogeneous evidence, legal, ethical, audit, and legitimacy assessments can feed a common closure calculation. The finite-stage results are supporting guarantees: they show that the interface evaluates deterministically and preserves ordinary T/F reasoning where no closure machinery is invoked.

The proposed calculus is conditional. Domain-specific evidential, legal, ethical, or institutional procedures supply typed atomic statuses and stopping designations. T-ACL then determines whether the compound governance claim is currently eligible for closure as acceptance, closure as rejection, or corrective non-closure.

The practical gap addressed here is a closure gap rather than a documentation gap. Contemporary AI governance instruments can require risk management, documentation, impact assessment, audit records, bias testing, and human oversight. Those instruments provide indispensable upstream inputs. T-ACL supplies the downstream interface that specifies how unresolved audit findings, remediable subgroup harm, legal uncertainty, and non-waivable legal failure jointly affect closure of a deployment claim.

T-ACL models this procedural status with T, F, and C. T indicates that closure as acceptance is currently permitted for the relevant institutional purpose. F indicates that closure as rejection is currently permitted. C indicates that neither acceptance nor rejection is yet procedurally eligible. A C-labelled claim may be evidentially underdetermined, evidentially conflicted, semantically unstable, ethically contested, or dependent on an unresolved subclaim. These triggers differ in source and remedy; they are grouped at the closure layer because they share one institutional consequence: final closure is not yet warranted.

This is why the calculus separates closure status from diagnostic reason. Evidence insufficiency, evidence conflict, semantic drift, audit failure, and ethical contestation do not have the same source and do not require the same remedy. They are represented by C only at the level of institutional closure. Their differences remain in the typed atomic status function, reason code, and audit record.

The three symbols belong to a closure-status algebra, not to an unrestricted truth-conditional semantics. If C were presented as a general truth value and if one asserted, without restriction, that $C \wedge F = C$, the objection would be immediate: in ordinary factual conjunction, if one conjunct is false, the conjunction is false. T-ACL therefore distinguishes two conjunctions:

- $\wedge_f$, factual conjunction, used for ordinary factual co-satisfaction;
- $\wedge_d$, deliberative or procedural conjunction, used for institutional closure conditions and dependency chains.

The difference is decisive. In the factual table, $C \wedge_f F = F$. In the deliberative table, $C \wedge_d F = C$. The deliberative reading is procedural: an unresolved corrective branch can prevent a compound review from being closed as a settled failure, unless the false branch is itself an unrevisable blocking condition.

The scope is prescriptive and downstream. Legal reasoning may determine that a rule is non-waivable; a fairness audit may identify remediable subgroup harm; a workflow system may record whether appeal paths are complete. T-ACL specifies how such outputs affect closure eligibility.

The central thesis is:

In finite-stage governance models, separating remediable non-closure from closure-level rejection and unrevisable blocking failure helps expose three failure modes: premature acceptance, premature rejection, and indefinite delay.

The formal apparatus below makes that claim explicit and checkable.

2 Existing Governance Tools and the Closure Gap

T-ACL is motivated by a gap between assurance inputs and closure decisions. Existing tools already cover much of the upstream terrain. Fairness toolkits such as AI Fairness 360 provide metrics and

mitigation methods for bias detection (Bellamy et al. 2019). Documentation practices such as model cards and datasheets improve transparency about model performance, intended use, data provenance, and limitations (Mitchell et al. 2019; Gebru et al. 2021). Human-centered design resources such as the People + AI Guidebook help teams make design and deployment choices more sensitive to users and stakeholders (Google PAIR n.d.). Risk frameworks such as the NIST AI RMF and regulatory instruments such as the EU AI Act organize governance obligations around risk, documentation, oversight, and conformity (NIST 2023; European Parliament and Council 2024).

These instruments are best understood as upstream sources for T-ACL rather than competitors. A model card may supply subgroup-performance evidence; a datasheet may supply provenance and collection limitations; a bias-audit tool may produce a remediable fairness finding; a legal review may identify a non-waivable prohibition; a workflow system may show that contestability fields are missing. The closure question remains: how should these heterogeneous inputs affect the deployment claim when some branches pass, some fail, and some remain unresolved?

T-ACL addresses that compositional question. Deferral, conditional approval, remediation, and escalation already occur in AI audit practice; the missing element is often a compact formal interface that separates deferral from rejection, remediable findings from final failure, valid hard stops from ordinary adverse findings, and complete audit records from incomplete closure files.

Auditability is treated as a well-formedness condition on the closure model. A C status must have a reason code, owner, expiry condition, next action, and appeal path. A stopping designation must have authority, scope, threshold, revocation condition, and role separation. Closure-relevant audit completeness or stopping authority is represented in the object language as an atom or as a valid stopping designation.

3 The Target Phenomenon: Corrective Non-Closure

The problem motivating T-ACL can be called corrective non-closure. A claim is in corrective non-closure when the available institutional procedure should not yet classify it as accepted or rejected, because the conditions for closure have not been satisfied. This status differs from ordinary ignorance in at least four ways.

First, it is actionable. A claim in C carries a procedure. That procedure may include additional data collection, adversarial review, expert consultation, public deliberation, model audit, ethical review, or legal scrutiny.

Second, it is auditable. The reason a claim remains open must be recorded with enough specificity to identify which evidence is missing, which evidence conflicts, which dependency remains unresolved, who is responsible for review, and when reassessment will occur.

Third, it is non-monotonic. A claim may move from T to C when new counter-evidence appears, from F to C when a failed branch becomes revisable, or from C to T/F when evidence thresholds are reached. This makes T-ACL closer to belief revision and dynamic update systems than to a static truth calculus.

Fourth, it is normatively constrained. Some procedures must not remain in C indefinitely. A C status can become a form of delay, evasion, or institutional capture. T-ACL therefore requires stopping rules, escalation rules, and audit rules. Long-term C is not success; it is a signal that the institution must either resolve the claim, acknowledge undecidability for present purposes, or revise the procedure.

3.1 Failure modes and design criteria

T-ACL is motivated by four recurring failure modes in AI governance. The first is premature acceptance: a deployment is approved because aggregate performance is acceptable while equity, audit, drift, or legal branches remain unresolved. The second is premature rejection: a compound claim is rejected before remediable or procedurally required review is completed. The third is indefinite delay: unresolved status becomes a way to avoid responsibility or postpone action without deadline. The fourth is captured stopping authority: a hard-stop designation is controlled by the party seeking approval or by an unaccountable office. These failures motivate the central design choices below: C blocks closure as acceptance while dependencies remain open; $\wedge_d$ separates remediable non-closure from closure-level rejection; C carries expiry, owner, action, and escalation conditions; and Stop_t requires role separation, public authority, scope, appeal path, and revocation conditions.

These criteria also clarify what would count as failure of the framework. If C can be renewed silently, T-ACL collapses into delay. If Stop_t can be declared by the audited party, it collapses into strategic veto.

If reason codes are omitted, C becomes an empty label. If factual conjunction and deliberative closure are not separated, the system invites a direct logical objection. The formal machinery below is built to block these failure modes rather than to maximize expressive power.

This target phenomenon is especially visible in AI governance. Consider an algorithmic hiring system. A claim such as "the system is fair enough for deployment" requires more than a single aggregate accuracy measure. The system may be accurate overall, inaccurate for a subgroup, under-audited with respect to data drift, or legally contested. A binary decision may conceal the unresolved structure. A C status records that the deployment claim is still non-closable.

The same structure appears in public policy. A transportation intervention may reduce average congestion but increase burdens on vulnerable groups. A public-health rule may be epidemiologically justified but legally or ethically contested. A climate policy may be empirically beneficial but distributionally unresolved. In these cases, C records suspended institutional closure.

4 Language

Let $\text{Atom} = \{p, q, r, \dots\}$ be a finite set of atomic claims. The language L_{TACL} is generated by:

$$\varphi ::= p \mid \neg\varphi \mid (\varphi \wedge_f \varphi) \mid (\varphi \wedge_d \varphi) \mid (\varphi \vee \varphi) \mid (\varphi \rightarrow \varphi).$$

The two conjunctions are primitive and distinct. There is no single ambiguous conjunction whose meaning is selected by a hidden mode parameter.

$\wedge_f$ is factual conjunction. It represents co-satisfaction of factual conditions. It is appropriate for claims such as "condition A is factually satisfied and condition B is factually satisfied."

$\wedge_d$ is deliberative conjunction. It represents procedural co-closure or dependency closure. It is appropriate for claims such as "all relevant branches of this governance claim are ready for closure" or "this decision may proceed only if each dependency is either accepted or legitimately closed."

This syntactic distinction is essential. A formula may contain both kinds of conjunction. For example, a governance claim may include a factual subclaim evaluated with $\wedge_f$ and an institutional closure condition evaluated with $\wedge_d$. The logic does not infer the intended interpretation from context after the fact; the modeller must encode it explicitly.

The modelling rule is simple:

- use $\wedge_f$ when the claim is about factual co-satisfaction of conditions;
- use $\wedge_d$ when the claim is about whether multiple governance dependencies are jointly ready for institutional closure.

For example, "the validation dataset covers region A and region B" is normally factual and should use $\wedge_f$. By contrast, "deployment may close only after performance, equity review, legal review, auditability, and drift review are all closure-ready" should use $\wedge_d$. A common modelling error is to encode legal compliance and unresolved auditability as $(\text{legal} \wedge_f \text{audit})$ and then treat the result as an ordinary factual subclaim. If auditability is a closure dependency, it must occur on an $\wedge_d$ closure path or be represented by a compound explicitly governed by Stop_t or an audit-status atom.

The status set is:

$$V = \{T, F, C\}$$

The elements of V are closure statuses. They use the familiar letters T, F, and C because the tables form a finite many-valued evaluation device, while the interpretation is institutional closure. A factual proposition may be supported upstream while the deployment claim containing it is C downstream because audit, appeal, legal, or remediation conditions remain open. Conversely, a deployment claim may be F for the present institutional purpose while some upstream factual or legal details remain under review.

The intended readings are:

- *T*: closure as acceptance is currently permitted for the present institutional purpose;
- *F*: closure as rejection is currently permitted for the present institutional purpose;
- *C*: corrective non-closure; neither acceptance nor rejection is currently eligible for final closure.

The phrase "for the present institutional purpose" is important. T-ACL is a status calculus for institutional acceptance, rejection, and suspended closure.

Three layers are involved. Upstream assessment asks what evidence, law, ethics, or argumentation supports. The closure-status layer asks whether a compound governance claim is closable as acceptance, closable as rejection, or still non-closable. Institutional action then decides whether to approve, reject,

suspend, remedy, escalate, disclose, or open appeal. T-ACL operates in the middle layer: it receives upstream statuses and supplies a structured closure result for institutional action.

The boundary between F and C is central. A negative finding does not automatically receive F. If the institution is still under a legitimate remediation window, if the relevant evidence is contested, or if the next required action is correction rather than final rejection, the branch is C with a negative audit reason. F is reserved for a branch or compound that may now be closed as rejected for the stated purpose. If that rejection is also non-waivable for the larger governance decision, Stop_t marks it as a blocking failure.

At the governance level this creates four distinct treatments. A claim accepted for the present purpose receives T and may close as acceptance. A remediable negative finding or unresolved dependency receives C and must remain open, be remedied, or be escalated. A closure-level rejection receives F and may close as rejection unless another unresolved branch prevents final compound closure. A non-waivable blocking failure receives F together with ValidStop_t , hence Block_t in a closure context, and closes the relevant deliberative compound as rejection.

Thus C is not a permission to delay indefinitely. It is a duty to continue, remedy, document, escalate, or terminate according to public stopping rules.

Expiry rule for C-status. Every C assignment must carry a public expiry condition: a deadline, maximum review cycle, or event-triggered reassessment condition. Silent renewal is invalid. Expiry leaves the three-valued status set unchanged; it is a governance transition constraint. If $\text{Val}_t(\phi) = C$, $\text{Expired}_t(\phi)$, and no $\text{ValidRenewal}_t(\phi)$ is recorded through an independent review path, then the institution must record $\text{CloseAction}_{t+1}(\phi)$ in $\{\text{reject}, \text{suspend}, \text{withdraw}\}$ or produce a valid time-limited renewal. For deployment claims, $\text{DeploymentPermitted}_{t+1}(\phi) = \text{false}$ until $\text{Val}_t(\phi) = T$ or a valid renewal is recorded.

Formally, T-ACL evaluates closure formulas over $V = \{T, F, C\}$. The reason-code vocabulary is a minimum working vocabulary for the first version of the calculus. It includes C_{evidence} for missing or low-quality evidence, C_{conflict} for active support of incompatible statuses, C_{drift} for semantic or distributional shift, C_{audit} for incomplete audit records or missing contestability, C_{ethics} for unresolved rights or affected-group objections, C_{legal} for legal uncertainty not yet resolved as binding failure, and $C_{\text{dependency}}$ for unresolved upstream subclaims. These are audit-level annotations rather than additional semantic values. Domain-specific implementations may refine the vocabulary while preserving the projection from reason-coded non-closure to C.

The calculus uses C for closure computation; Audit_t preserves the diagnostic reason, responsible unit, expiry condition, and next action.

Reason codes are handled by a separate partial function:

$$\text{Reason}_t(\phi) \in R, \quad R = \{C_{\text{evidence}}, C_{\text{conflict}}, C_{\text{drift}}, C_{\text{audit}}, C_{\text{ethics}}, C_{\text{legal}}, C_{\text{dependency}}\}.$$

$\text{Reason}_t(\phi)$ is defined when $\text{Val}_t(\phi) = C$, or when the audit log records a diagnostic reason for a status transition. The precise representation is $\text{Val}_t(\phi) = C$ together with $\text{Reason}_t(\phi) = C_{\text{legal}}$; the reason code is a diagnostic annotation carried by the audit layer.

4.1 Projection from diagnostic reasons to closure status

The relation between diagnostic detail and closure status is a two-level semantics. The diagnostic space is the reason-code set R. The closure space is the status set V. T-ACL uses a closure projection from R to V: every active diagnostic reason in R projects to C for the purpose of closure computation, while the reason itself remains available in Reason_t and Audit_t .

The projection principle is:

- if $\text{Reason}_t(\phi) = r$ for some active diagnostic reason r in R, then $\text{Val}_t(\phi) = C$ for closure evaluation unless a valid non-waivable F plus Stop_t has been established;
- the reverse implication does not erase the reason: every C used for closure must be backed by a reason, owner, expiry condition, and next action;
- a C assignment without a reason code or audit record is ill-formed for governance use.

This projection semantics is the answer to the coarse-granularity objection. C is coarse only at the closure layer. It is not coarse at the diagnostic layer, because the diagnostic layer preserves why the claim is non-closable and what institutional action follows. Without this projection discipline, C would collapse into an empty label. With it, C is an auditable compression from heterogeneous governance reasons to a single closure consequence.

Well-formed C assignment. A C assignment is well-formed only if the audit layer defines the following fields:

- $\text{Reason}_t(\phi)$;
- $\text{Owner}_t(\phi)$;
- $\text{Expiry}_t(\phi)$;
- $\text{NextAction}_t(\phi)$;
- $\text{AppealPath}_t(\phi)$.

$\text{Reason}_t(\phi)$ states why the claim is non-closable. $\text{Owner}_t(\phi)$ identifies the responsible unit. $\text{Expiry}_t(\phi)$ gives the deadline, review cycle, or event trigger. $\text{NextAction}_t(\phi)$ specifies the required remedy, review, escalation, suspension, or withdrawal step. $\text{AppealPath}_t(\phi)$ states how affected parties or reviewers may contest the status. These functions may be implemented as fields inside Audit_t ; they are written separately here to make the well-formedness condition explicit.

If $\text{Val}_t(\phi) = C$ and any of these fields is missing, the model is ill-formed for governance use. The status may still be syntactically computable, but it is not an auditable T-ACL closure model. This condition is what prevents C from becoming a vague middle label or a silent delay state.

T-ACL also uses stopping designations with two levels. A proposed stop, written as ProposedStop_t , records that some institution or review process claims that a formula, if F, should block closure. A valid stop, written as ValidStop_t , means that the proposed stop has passed the authority, scope, evidence-threshold, appeal, revocation, and role-separation tests required for the relevant procedure. When no ambiguity is possible, Stop_t is used as shorthand for ValidStop_t .

Examples of valid stops include a binding legal prohibition, a non-waivable safety failure, an ethical red line, or an audit condition without which deployment is prohibited. A proposed stop becomes operative only after validation. If the designation is contested, the designation itself becomes a claim subject to review and triggers blocking only after validation.

The legitimacy of ValidStop_t comes from the relevant legal, safety, ethical, or institutional authority and must be recorded in Audit_t . T-ACL exposes where such authority is invoked and makes its procedural trace visible inside the closure model.

A valid stopping designation must be reviewable. Its audit record must state the authority source, scope condition, evidence threshold, appeal path, revocation condition, and abuse-risk note. If these fields are missing, $\text{ProposedStop}_t(\phi)$ may be recorded, but $\text{ValidStop}_t(\phi)$ is false or unresolved. The authority of the stopping condition itself should then be represented as C, usually with reason code C_{legal} , C_{ethics} , or C_{audit} .

Role-separation principle. For ValidStop_t , the stopping authority, operational closure body, audited party, and audit or appeal body must be functionally separated. A stopping designation controlled by the audited party or by the same unit seeking approval is invalid unless independently ratified and contestable. If role separation is not satisfied, the authority-validity atom for the stopping designation should be C, usually C_{audit} or C_{legal} ; blocking starts only after independent authority is established. This can be encoded directly. Let h mean that the authority source for the stopping designation is procedurally valid. A deployment claim may then be represented as:

$$G = (((h \wedge d) \wedge a) \wedge s)$$

If $\text{Val}_t(h) = C$ with $\text{Reason}_t(h) = C_{\text{legal}}$ or $\text{Reason}_t(h) = C_{\text{audit}}$, $\text{ProposedStop}_t(\ell)$ is not yet valid and the result remains C. Only when $\text{Val}_t(h) = T$, $\text{ValidStop}_t(\ell)$, and $\text{Val}_t(\ell) = F$ may the legal failure propagate through Block_t .

Valid-stop condition. Strictly speaking, there is a difference between a proposed stopping designation and a valid stopping designation. A proposed designation is merely an institutional claim that a formula should block closure if false. A valid designation satisfies the metadata and role-separation requirements above. Below, $\text{Stop}_t(\phi)$ means valid stopping designation unless explicitly qualified. Invalid or contested stopping designations do not propagate. Formally, if the authority-validity atom h_ϕ for a proposed stop is C or F, then $\text{LocalBlock}_t(\phi)$ is not triggered even when $\text{Val}_t(\phi) = F$; the compound remains C or follows the ordinary table until the authority question is resolved.

Equivalently:

$$\text{LocalBlock}_t(\phi) \iff \text{ValidStop}_t(\phi) \wedge \text{Val}_t(\phi) = F.$$

This rule blocks a serious abuse path. A deployment operator must not be allowed to define its own hard stop, and an approval office must not be allowed to weaponize a contested hard stop to avoid review. Stop validity requires independent authority, scope, evidence threshold, appeal path, and revocation condition.

ValidStop_t does not change the factual or evidential status of ϕ itself. It changes only how a false ϕ propagates inside deliberative closure formulas. The derived predicate $\text{Block}_t(\phi)$ means that ϕ contains, or is itself, an already triggered unrevisable blocking failure. Block_t is recursive and is used to ensure that a legal, safety, or ethical blocking condition is not lost when larger $\wedge_d$ formulas are evaluated.

5 Finite-Stage Semantics

A finite-stage T-ACL model is:

$$M_t = (\text{Atom}, E_t, \alpha_t, \text{ProposedStop}_t, \text{ValidStop}_t, \text{Block}_t, \text{Val}_t, \text{Audit}_t, \text{Reason}_t)$$

where:

- Atom is a finite set of atomic claims;
- $E_t(p)$ is the evidence set for atom p at stage t ;
- $\alpha_t(p)$ is a public typed atomic status function returning a value in V ;
- ProposedStop_t records proposed non-waivable blocking designations at stage t ;
- ValidStop_t records those proposed designations that satisfy authority, scope, threshold, appeal, revocation, and role-separation conditions;
- Block_t is the recursive blocking predicate induced by ValidStop_t and Val_t ;
- Val_t maps formulas to statuses in V ;
- Audit_t records evidence sources, thresholds, updates, authority, expiry conditions, and closure decisions;
- Reason_t records audit-level diagnostic reason codes when $\text{Val}_t(\phi) = C$ or when a status transition requires explanation.

For atoms, the status is supplied by the typed atomic status function:

$$\text{Val}_t(p) = \alpha_t(p)$$

Reason codes do not enlarge the semantic status set. They are audit annotations used to explain why a formula received C or why a transition occurred. A phrase such as C_{legal} is shorthand for $\text{Val}_t(\phi) = C$ and $\text{Reason}_t(\phi) = C_{\text{legal}}$.

The distinction is semantic rather than notational. C is mathematically a value of the finite status algebra V ; it is not interpreted as an unrestricted truth value for all propositions. T , F , and C are closure statuses assigned to review claims at a fixed institutional stage. T-ACL can therefore be compared with K3, LP, and Belnap-style systems as a many-valued formal device, while retaining a procedural closure interpretation.

For factual and predictive atoms, α_t may be implemented by thresholding a public evidence score:

$$\alpha_t(p) = \begin{cases} T & \text{if } \Theta_t(p) \geq \tau_T, \\ F & \text{if } \Theta_t(p) \leq -\tau_F, \\ C & \text{otherwise.} \end{cases}$$

For ethical, legal, and legitimacy atoms, α_t may include veto filters, public-reason requirements, contestability checks, or stopping designations. Thus the threshold rule is a special case, not the general semantics of atomic status. The finite-stage semantics assumes only that, for the present stage, atomic statuses have been publicly supplied by α_t and recorded or contestable through Audit_t and Reason_t .

Composite formulas are evaluated by tables.

5.1 Negation

Negation is defined by **not** $T = F$, **not** $F = T$, and **not** $C = C$. Negating an unresolved corrective claim does not resolve it. If a claim is under review, its negation is also under review unless independent evidence settles the negated form.

5.2 Factual conjunction

$\wedge_f$	T	C	F
T	T	C	F
C	C	C	F
F	F	F	F

Here $C \wedge_f F = F$. This is the ordinary factual intuition: a false conjunct is enough to make the factual conjunction fail.

5.3 Deliberative conjunction

$\wedge_d$	T	C	F
T	T	C	F
C	C	C	C
F	F	C	F

Here $C \wedge_d F = C$. This is a closure rule for compound governance decisions. The reading is: one branch is already eligible for rejection, but another dependency is still unresolved; therefore the compound is not yet eligible for final closure unless the failing branch is non-waivable. This prevents an institution from using an ordinary remediable or branch-level failure to avoid resolving a still-open audit, equity, legality, or drift dependency. If the F branch is non-waivable, Stop_t and Block_t override the ordinary table and force the deliberative compound to F.

The table is the operator for the specific institutional policy defended here: unresolved dependencies remain visible when some other branch currently fails, while non-waivable failures propagate through the stopping mechanism. Other governance settings may use stricter or more permissive deliberative operators, but they should be named separately under different connectives.

Three policy choices are available. A strict rejection operator would set $C \wedge_d F = F$, protecting against delay but risking premature rejection before unresolved review duties are satisfied. A permissive delay operator would allow almost any C to keep a compound open, protecting unresolved review but risking indefinite suspension. T-ACL adopts the middle option: $C \wedge_d F = C$ unless the F branch is blocking. This keeps unresolved dependencies visible while preserving hard stops.

The governance rationale can be stated as a minimax choice among two institutional errors. A strict rejection operator protects against delay but can hide unfinished review duties: for example, a remediable performance failure could close the entire deployment as rejected before a pending discrimination complaint is reviewed, thereby erasing the rights-based audit trail. A permissive delay operator protects unresolved review but can hide binding failure: for example, a legal prohibition could be kept inside C indefinitely. T-ACL's guarded $\wedge_d$ chooses the middle policy. Ordinary remediable failure does not erase unresolved dependencies, but a valid non-waivable failure propagates through Stop_t and Block_t .

This table expresses a normative design choice for deliberative closure formulas whose purpose is to decide whether an institution may close a compound governance claim. A modeller who wants strict rejection or permissive delay must name a different operator instead of changing the meaning of $\wedge_d$.

In the absence of triggered blocking, $\wedge_d$ is commutative and associative on the three displayed statuses, with T as the identity and C as an absorbing status relative to mixed unresolved/failing dependencies. Triggered Block_t changes the evaluation because it is not an ordinary truth-functional value; it is a closure-level guard. Thus algebraic claims about $\wedge_d$ apply to the unguarded table, while guarded closure formulas must be evaluated by Algorithm 1.

The unguarded table can be characterized by five closure postulates:

- **Acceptance identity.** Combining a closure-ready branch with another branch should not change the latter's status: $T \wedge_d X = X$.
- **Dependency preservation.** If any non-blocking dependency is C, the compound remains C until the dependency is resolved or lawfully escalated.
- **Settled rejection without unresolved dependencies.** If no branch is C and at least one branch is F, the compound is F.
- **Commutativity and associativity.** The order and bracketing of independent closure dependencies should not affect the unguarded status.
- **Hard-stop priority.** A valid non-waivable F is not handled by the unguarded table; it is handled by ValidStop_t and Block_t .

These postulates determine the displayed $\wedge_d$ table on V once triggered blocking is excluded. If any input is C, Postulate 2 gives C; if no input is C but at least one input is F, Postulate 3 gives F; otherwise all inputs are T and Postulate 1 gives T. The result is not derived from ordinary conjunction. It is an explicit policy for closure dependencies.

The table is guarded by the recursive predicate Block_t . The predicate is defined bottom-up:

- $\text{LocalBlock}_t(\phi)$ holds exactly when $\text{ValidStop}_t(\phi)$ and $\text{Val}_t(\phi) = F$;
- $\text{Block}_t(\phi)$ holds if $\text{LocalBlock}_t(\phi)$ holds;
- $\text{Block}_t(\phi \wedge_d \psi)$ holds if $\text{LocalBlock}_t(\phi \wedge_d \psi)$, $\text{Block}_t(\phi)$, or $\text{Block}_t(\psi)$ holds;

- for the other connectives, child blocking does not automatically propagate to the compound.

For a compound $(\phi \wedge_d \psi)$, if $\text{Block}_t(\phi)$ or $\text{Block}_t(\psi)$, the evaluation returns (F, true) .

Otherwise the ordinary $\wedge_d$ table above applies. This prevents $\wedge_d$ from becoming a formal excuse for delay. A currently negative but remediable empirical branch should be represented as C with an audit reason, not as F, while the remediation window remains open. An unrevisable legal, safety, or ethical blocking condition terminates closure and propagates upward through the deliberative closure formula.

5.4 Disjunction

Disjunction is the maximum operation under the ordering $F < C < T$: if either disjunct is T, the result is T; if neither is T but at least one is C, the result is C; only $F \vee F$ is F. This is a binary status computation when both disjuncts have known statuses. The system does not adopt a status-preserving rule that introduces a disjunction from a single premise by treating the missing disjunct as F. Such a rule is unsound for exact status computation.

5.5 Implication

Implication is defined as follows: if the antecedent is F, the implication is T; if the antecedent is T, the implication inherits the consequent's status; if the antecedent is C, the implication is C. This preserves the classical idea that a false antecedent makes material implication acceptable, while an unresolved antecedent prevents determinate closure. T-ACL uses this implication only as a finite status connective. It is not a full deontic, causal, or counterfactual implication. Governance rules involving duties, permissions, legal exceptions, or defeasible priorities should be represented upstream by legal or deontic machinery and then passed to T-ACL through typed atoms or stopping designations.

6 Pair-Labelled Calculation Rules

Judgements have the form:

$$\vdash_t \phi : (X, b)$$

where X is one of T , F , or C , and b is a Boolean blocking label. The judgement means that, in the current finite stage, formula ϕ has status X and blocking status b .

The rule system is table-driven and pair-labelled. It is closer to a labelled calculation calculus than to an ambitious proof theory. It computes the same pair as Eval_t : the first component is the value/status, and the second component records whether Block_t holds. Thus blocking is not a semantic oracle external to the calculus; it is an explicit part of the derived judgement.

6.1 Atomic rule

If the typed atomic status function assigns $\alpha_t(p) = X$, infer:

$$\vdash_t p : (X, b)$$

where $b = \text{true}$ iff $\text{ValidStop}_t(p)$ and $X = F$.

6.2 Negation rule

If $\vdash_t \phi : (X, b_\phi)$, infer:

$$\vdash_t \neg\phi : (Y, b)$$

where Y is given by the negation table, and $b = \text{true}$ iff $\text{ValidStop}_t(\neg\phi)$ and $Y = F$.

6.3 Factual conjunction rule

If $\vdash_t \phi : (X, b_\phi)$ and $\vdash_t \psi : (Y, b_\psi)$, infer:

$$\vdash_t (\phi \wedge_f \psi) : (Z, b)$$

where $Z = \wedge_f(X, Y)$, and $b = \text{true}$ iff $\text{ValidStop}_t(\phi \wedge_f \psi)$ and $Z = F$.

6.4 Deliberative conjunction rule

If $\vdash_t \phi : (X, b_\phi)$ and $\vdash_t \psi : (Y, b_\psi)$, infer:

$$\vdash_t (\phi \wedge_d \psi) : (Z, b)$$

where if b_ϕ or b_ψ , then $Z = F$ and $b = \text{true}$. Otherwise $Z = \wedge_d(X, Y)$, and $b = \text{true}$ iff $\text{ValidStop}_t(\phi \wedge_d \psi)$ and $Z = F$.

6.5 Disjunction rule

If $\vdash_t \phi : (X, b_\phi)$ and $\vdash_t \psi : (Y, b_\psi)$, infer:

$$\vdash_t (\phi \vee \psi) : (Z, b)$$

where $Z = \vee(X, Y)$, and $b = \text{true}$ iff $\text{ValidStop}_t(\phi \vee \psi)$ and $Z = F$.

No exact-status rule is provided for inferring $(\phi \vee \psi)$ from only one established disjunct, except in derived weak forms explicitly marked as non-exact. This avoids the error of assigning an unknown second disjunct a default value.

6.6 Implication rule

If $\vdash_t \phi : (X, b_\phi)$ and $\vdash_t \psi : (Y, b_\psi)$, infer:

$$\vdash_t (\phi \rightarrow \psi) : (Z, b)$$

where $Z = \Rightarrow(X, Y)$, and $b = \text{true}$ iff $\text{ValidStop}_t(\phi \rightarrow \psi)$ and $Z = F$.

6.7 Conflict consolidation rule

If independent evidence channels produce active support for both $\vdash_t \phi : (T, b_T)$ and $\vdash_t \phi : (F, b_F)$, the system does not infer arbitrary χ . Instead, the conflict is consolidated into:

$$\vdash_{t+1} \phi : (C, \text{false})$$

for the next stabilized stage, together with an audit record. This rule is best understood as a stage-update rule, not as a static object-level theorem. After consolidation, the conflicting T and F labels are not jointly available as unrestricted inferential resources.

This point is crucial. If conflict consolidation withdraws or suspends prior labels, the system is non-monotonic. T-ACL should therefore be read as a finite-stage status calculus with update operations, not as a monotonic classical consequence relation.

7 Formal Definitions

Definition 1 (Finite-stage model). A finite-stage T-ACL model is a tuple:

$$M_t = (\text{Atom}, E_t, \alpha_t, \text{ProposedStop}_t, \text{ValidStop}_t, \text{Block}_t, \text{Val}_t, \text{Audit}_t, \text{Reason}_t)$$

where all components are finite and public at stage t , atomic statuses are supplied by the typed function α_t , ProposedStop_t records asserted stopping designations, ValidStop_t records those designations that have passed authority and role-separation checks, Block_t is recursively induced by ValidStop_t and status evaluation, Reason_t stores audit-level diagnostic annotations, and composite formulas are evaluated by the tables in Section 4 together with the Block_t guard for $\wedge_d$.

Definition 2 (Local and propagated blocking). $\text{LocalBlock}_t(\phi)$ and $\text{Block}_t(\phi)$ are defined as follows:

- if $\text{ValidStop}_t(\phi)$ and $\text{Val}_t(\phi) = F$, then $\text{LocalBlock}_t(\phi)$;
- if $\text{LocalBlock}_t(\phi)$, then $\text{Block}_t(\phi)$;
- if $\text{Block}_t(\phi)$ or $\text{Block}_t(\psi)$, then $\text{Block}_t(\phi \wedge_d \psi)$;
- if $\text{LocalBlock}_t(\phi \wedge_d \psi)$, then $\text{Block}_t(\phi \wedge_d \psi)$;
- no further blocking facts are inferred unless explicitly generated by these clauses.

The least-predicate condition matters: local blocking may be declared for any formula, but propagated blocking travels upward only through deliberative closure. It does not leak through unrelated truth-functional connectives.

The following equivalent evaluation procedure removes any apparent circularity between Val_t and Block_t .

Algorithm 1 (Joint finite-stage evaluation). $\text{Eval}_t(\phi)$ returns a pair (v, b) , where v is the status of ϕ and b is whether $\text{Block}_t(\phi)$ holds.

The clauses are structural:

- Atom: $\text{Eval}_t(p)$ returns $(\alpha_t(p), \text{LocalBlock}_t(p))$.
- Negation: if $\text{Eval}_t(\phi) = (v, b)$, then $\text{Eval}_t(\neg\phi)$ returns $(\text{not_table}(v), \text{LocalBlock}_t(\neg\phi))$.
- Factual conjunction: evaluate both children, apply the factual conjunction table, then apply the local-block check to the compound.
- Deliberative conjunction: evaluate both children. If either child is blocked, return (F, true) ; otherwise apply the deliberative conjunction table and then the local-block check.
- Disjunction: evaluate both children, apply the disjunction table, then apply the local-block check.
- Implication: evaluate both children, apply the implication table, then apply the local-block check.

In each composite clause, LocalBlock_t is evaluated using the status z just computed for that same compound. Thus the algorithm is structural and does not require a separate fixed-point computation.

For connectives other than $\wedge_d$, an internal blocking fact does not automatically propagate to the compound unless the compound itself is locally blocked. This is a design choice: propagated blocking is tied to deliberative closure, not to all truth-functional composition.

This design imposes a well-formedness requirement on closure models. If a non-waivable condition is intended to control institutional closure, it must either occur on an $\wedge_d$ closure path or the smallest compound through which it is meant to control closure must itself be designated by Stop_t . For instance, if ℓ is a non-waivable legal condition inside $(a \wedge_f \ell) \wedge_d s$, and the modeller intends the legal failure to block the whole closure decision, then either ℓ must be placed as an explicit $\wedge_d$ dependency or $\text{Stop}_t(a \wedge_f \ell)$ must be declared. Otherwise T-ACL treats the loss of propagated blocking as a modelling error, not as a semantic permission to bypass the legal constraint. This condition prevents blocking facts from being accidentally erased by factual or material connectives while preserving the principle that blocking propagation is a closure-level mechanism.

Lemma 1 (Existence and uniqueness of finite-stage evaluation). *For every finite formula ϕ and fixed finite-stage model M_t , $\text{Eval}_t(\phi)$ returns a unique pair $(\text{Val}_t(\phi), \text{Block}_t(\phi))$.*

Proof. The proof is by structural induction on ϕ . Atomic formulas are evaluated by the public typed status function α_t and the local stopping designation, so the pair is unique. Each composite clause first evaluates strictly smaller subformulas and then applies a deterministic table and a deterministic local-blocking check. Since the syntax tree is finite, the recursion terminates and the returned pair is unique. QED.

Lemma 2 (Blocking conservativity). *If a formula contains no $\wedge_d$, or if no subformula occurring in an $\wedge_d$ chain triggers Block_t , then blocking propagation does not alter the ordinary table value of the formula.*

Proof. Inspect Algorithm 1. The only clause that propagates a subformula's blocking value to the parent is the $\wedge_d$ clause. All other connectives compute their status from the ordinary table and check only whether the compound itself is locally blocked. Therefore, in formulas without deliberative closure, or in deliberative formulas without triggered blocking subformulas, Block_t does not change the table-computed status. QED.

Definition 3 (Exact pair derivability). $\vdash_t \phi : (X, b)$ means that formula ϕ is derivable with exact status X and blocking label b at stage t from the atomic status assignment, connective tables, stopping designations, and pair-labelled rules of M_t . Exact pair derivability is not ordinary assertion. It is a labelled computation of the current procedural status and closure-blocking status.

Definition 4 (Stage transition). A stage transition from M_t to M_{t+1} may update evidence sets, typed atomic status functions, thresholds, stopping designations, reason codes, expiry records, closure actions, or audit records. A transition is admissible only if the update is recorded in Audit_{t+1} .

If active support for both T and F is detected for a formula at stage t , the transition may consolidate the formula to C at stage $t+1$ and record the conflicting sources. The prior T and F labels are then withdrawn or suspended for exact-status inference at the stabilized stage.

If $\text{Val}_t(\phi) = C$, $\text{Expired}_t(\phi)$, and not $\text{ValidRenewal}_t(\phi)$, the transition must record a governance action:

$$\text{CloseAction}_{t+1}(\phi) \in \{\text{reject}, \text{suspend}, \text{withdraw}\}$$

For deployment claims, the transition also records:

$$\text{DeploymentPermitted}_{t+1}(\phi) = \text{false}$$

until $\text{Val}_t(\phi) = T$ or a valid independent renewal is recorded. This expiry constraint does not alter the semantic value set $V = \{T, F, C\}$; it specifies what the institution must do when corrective non-closure exceeds its public review window.

Definition 5 (Classical fragment). The classical fragment is the subset of L_{TACL} using only atoms assigned T or F, together with $\neg$, $\wedge_f$, $\vee$, and $\rightarrow$. No occurrence of C is permitted in the atomic assignment, and no formula uses $\wedge_d$, ProposedStop_t , ValidStop_t , Block_t , or stage-update rules.

These definitions make the scope of the formal results explicit. The propositions below are design guarantees for a finite closure-status layer. Their role is modest but important: once a governance model has supplied typed atomic statuses and stopping designations, the closure layer behaves deterministically, preserves the ordinary T/F fragment, and treats conflict as a review trigger rather than arbitrary inference.

8 Closure-Control Separation

The main design property is that T-ACL separates two governance cases that look similar if all negative findings are simply marked as false.

Proposition 1 (Closure-control separation). *Let $G = (x \wedge_d y)$ be a deliberative closure formula. Suppose $x = T$ and y is a negative finding. T-ACL can represent two distinct institutional situations. If the finding is remediable or still under review, encode $y = C$ with an audit reason such as C_{ethics} , C_{evidence} , or C_{audit} ; then $\text{Eval}_t(G) = C$. If the finding is a valid non-waivable blocking failure, encode $y = F$ and declare valid $\text{Stop}_t(y)$; then $\text{Eval}_t(G) = (F, \text{true})$.*

Proof. In the first case, the unguarded deliberative conjunction table gives $T \wedge_d C = C$, so the compound remains non-closable and the audit reason determines the next required action. In the second case, $\text{Stop}_t(y)$ and $\text{Val}_t(y) = F$ make $\text{LocalBlock}_t(y)$ true. Algorithm 1 propagates the blocking label through the surrounding $\wedge_d$ formula and returns (F, true) . Thus the same kind of negative evidence can be represented either as remediable corrective non-closure or as a non-waivable blocking failure, depending on the public authority and audit record. QED.

This is the core formal contribution: the third status is tied to distinct closure-control paths that ordinary binary approval systems tend to collapse.

9 Finite-Stage Design Guarantees

The remaining formal results are supporting checks for the interface. They establish predictable fixed-stage behavior once typed atomic statuses and valid stopping designations have been supplied. Their function is to support the closure-control separation by ruling out ambiguity in the calculus that implements it.

Proposition 2 (Finite-stage correctness). *For a fixed finite-stage model M_t , the pair-labelled calculus and Algorithm 1 assign the same pair to every finite formula: $\vdash_t \phi : (X, b)$ exactly when $\text{Eval}_t(\phi) = (X, b)$.*

Proof. Both directions are structural inductions on ϕ or on the labelled derivation. Atomic formulas use α_t and the local stopping condition. Each connective rule applies the same table as Algorithm 1; the $\wedge_d$ case also applies the same child-blocking guard. Since Lemma 1 gives a unique finite evaluation pair, the derivation and the algorithm coincide. QED.

Proposition 3 (Classical-fragment conservativity). *On formulas built only from T/F-valued atoms, $\neg$, $\wedge_f$, $\vee$, and $\rightarrow$, T-ACL agrees with ordinary propositional evaluation.*

Proof. In the restricted fragment, C , $\wedge_d$, stopping designations, blocking, and stage-update rules do not occur. The remaining tables reduce to the classical truth tables, and structural induction gives the result. QED.

Proposition 4 (Procedural conflict consolidation). *Active T/F conflict for a claim does not license arbitrary conclusions. It creates a recorded transition to C at the next stabilized stage.*

Proof. The rule set contains no explosion rule. Conflict consolidation records the conflicting support and assigns the claim a corrective status at the next stage. The claim then functions as a review trigger, not as an unrestricted inferential resource. QED.

For implementation, the important complexity fact is fixed-stage reproducibility: for a finite formula of length n , evaluation is linear in n once α_t and valid stopping designations are supplied. The abstract satisfiability variant inherits the familiar cost profile of propositional satisfiability, because the ordinary T/F fragment embeds classical SAT and candidate atomic assignments can be checked in polynomial time. This observation delimits the formal object. In applied settings, the relevant cost and contestability lie upstream, in the procedures that generate α_t , validate stopping authority, and produce audit records.

10 F/C Boundary Tests

The most important modelling boundary is not T versus C, but C versus F. A negative finding can mean at least four different things. T-ACL treats them differently:

Boundary case	Coding	Why
remediable negative finding	$\text{Val}_t(\phi) = C$, with a negative reason code such as C_{ethics} , C_{audit} , or C_{evidence}	the branch is adverse but still under a legitimate corrective process
closure-level rejection	$\text{Val}_t(\phi) = F$, no valid stop	the branch may be closed as failed for the present purpose, but it is not yet a non-waivable block on a larger deliberative compound
non-waivable legal, safety, or rights failure	$\text{Val}_t(\phi) = F$ plus $\text{ValidStop}_t(\phi)$, hence $\text{Block}_t(\phi)$ in a closure context	the failure is not merely adverse evidence; it terminates the relevant closure path
expired C without valid renewal	requires recorded action in $\{\text{reject}, \text{suspend}, \text{withdraw}\}$ or role-separated renewal	C is a time-limited review status

These tests discipline the use of $\wedge_d$. If a modeller wants a negative branch to remain C, the record must show a reason, owner, expiry, next action, and review path. If a modeller wants a negative branch to become blocking F, the record must show authority, scope, evidence threshold, appeal or contestation path, revocation condition, and role separation. Without these metadata, the model is ill-formed for governance use.

11 Typed Atomic Status Functions and Audit Logs

The typed atomic status function α_t is the boundary of the calculus. A well-formed governance model admits α_t only when the status-generating procedure is public, typed, authority-linked, and auditable. Hidden or unreviewable status generation makes the closure model ill-formed for governance use. For factual and predictive claims, one common component of α_t is a public evidence score:

$$\Theta_t(\phi) = \text{weighted aggregate of signed evidence items}$$

where evidence items include source, timestamp, support direction, quality score, known bias risk, and method of collection. In those cases, α_t may threshold Θ_t using τ_T and τ_F .

However, a single weighted average is not sufficient for all atoms. T-ACL distinguishes at least five claim types:

1. factual claims; 2. predictive claims; 3. normative or ethical claims; 4. legal claims; 5. institutional legitimacy claims.

Factual and predictive claims may use statistical evidence, models, confidence intervals, and historical comparisons. Normative claims require deliberative input, rights constraints, and minority-protection checks. Institutional legitimacy claims require legality, authority, procedural fairness, and contestability.

A general typed atomic interface is therefore:

$$\alpha_t(\phi) = G_{\text{type}}(E_{\text{data}}, E_{\text{expert}}, E_{\text{public}}, E_{\text{model}}, E_{\text{history}}, R_{\text{ethics}}, L_{\text{legal}}, \text{Audit}_t).$$

where G_{type} depends on the claim type, R_{ethics} is an ethical filter, and L_{legal} records binding legal constraints. A claim cannot become institutionally acceptable merely because a numerical score is high if it fails rights-based or legal constraints.

A minimal typed status procedure has five clauses. First, if the audit record is incomplete for a closure-relevant claim, the atom receives $\text{Val}_t(\phi) = C$ with $\text{Reason}_t(\phi) = C_{\text{audit}}$. Second, if a legal claim has a binding non-waivable failure, the atom receives F and a public stopping designation $\text{Stop}_t(\phi)$ is recorded. Third, if an ethical claim has an open remedy process, the atom receives $\text{Val}_t(\phi) = C$ with $\text{Reason}_t(\phi) = C_{\text{ethics}}$. Fourth, if a factual or predictive claim passes the public threshold and stability checks, it receives T; if it falls below the rejection threshold and no remediation window remains, it receives F; otherwise it receives $\text{Val}_t(\phi) = C$ with $\text{Reason}_t(\phi) = C_{\text{evidence}}$. Fifth, unresolved upstream dependencies receive $\text{Val}_t(\phi) = C$ with $\text{Reason}_t(\phi) = C_{\text{dependency}}$.

The pseudocode specifies the minimal interface expected by the closure calculus: each atom must arrive with a status, a reason code, and an audit trail sufficient for contestation. A closure model whose upstream procedure is undocumented, untyped, or immune from review is ill-formed.

Opaque-input rule. If $\alpha_t(\phi)$ is closure-relevant and the model cannot identify the status-generating authority, evidence basis, claim type, review path, and contestation channel for ϕ , then ϕ may not be used as T or F in a well-formed governance model. The modeller must either assign $\text{Val}_t(\phi) = C$ with $\text{Reason}_t(\phi) = C_{\text{audit}}$ or represent input opacity as a separate audit atom on the deliberative closure path. This rule keeps unreviewable upstream labels outside valid closure.

The typed status generation protocol is:

1. identify the claim type; 2. declare the competent status-generating authority; 3. record evidence sources and thresholds before evaluation; 4. specify T, C, F, and Stop eligibility conditions; 5. assign owner, expiry, next action, and appeal path for every C; 6. record role separation when a Stop designation may be invoked; 7. log dissenting evidence or contested authority.

Two validity atoms are useful in implementations. Let h_ϕ record whether the authority for a proposed stopping designation on ϕ is valid and role-separated. Let u_ϕ record whether the typed atomic status procedure for ϕ is public and reviewable. If $u_\phi = C$, the affected closure formula remains C because the status input is not yet reviewable. If $u_\phi = F$ and reviewability is non-waivable for the decision, valid $\text{Stop}_t(u_\phi)$ may block closure. Opacity therefore becomes a closure-relevant atom rather than a hidden premise.

For implementation, the evidence interface should be read as a typed schema rather than a universal scalar epistemology. Factual and predictive atoms normally require measurements, records, validation data, drift tests, stability checks, and public thresholds. Ethical atoms require rights-impact information, affected-group objections, remedy paths, and review authority. Legal atoms require a competent source of legal authority, scope, citation, and appeal path. Audit atoms require provenance, source, owner, contestability, and expiry information. Institutional-legitimacy atoms require mandate, authorization, role separation, and public reason. The same typed protocol also governs rejection and stopping: remediable failures produce C with a reason code, closure-level failures produce F, and non-waivable failures require a valid stopping designation before they propagate as Block_t .

Thus R_{ethics} is not normally a small additive term. In many governance settings it is a filter or veto condition: it may force C while remedy is open, or F through ValidStop_t and Block_t when a non-waivable right, safety, or legal constraint is violated. Public contestation is not automatically evidence that a claim is false; it is evidence that legitimacy, auditability, or public-reason conditions may be unresolved.

Thresholds τ_T and τ_F must be set before the relevant evaluation stage, recorded in Audit_t , and justified by the competent authority. Threshold authority is part of the institutional precondition for a well-formed typed status function.

Every transition from C to T or F must record the evidence used, decision authority, time, dissenting evidence, and appeal or review mechanism. The audit log is the institutional condition that makes typed status assignment contestable.

For a minimal implementation, an audit record is complete only if it identifies the claim and stage, records status and reason, links the status to a source and authority, assigns an owner and expiry condition, and records appeal and role-separation checks. If required fields are absent for a closure-relevant claim, the audit layer should assign $\text{Val}_t(\phi) = C$ with $\text{Reason}_t(\phi) = C_{\text{audit}}$. If audit completeness is non-waivable, the audit-status atom may instead be assigned F with $\text{ValidStop}_t(\text{audit_complete})$.

If audit completeness is itself a closure condition, it must be represented in the object language rather than left as an informal background assumption. For example, let a mean "the audit record is complete, public, and contestable." If $a = C$, a deliberative closure formula containing a remains non-closable. If $a = F$ and audit completeness is non-waivable for the relevant deployment or policy decision, then $\text{ValidStop}_t(a)$ is declared; by Algorithm 1, $\text{Block}_t(a)$ propagates through the surrounding $\wedge_d$ closure formula and the compound is rejected. This is the formal bridge between Audit_t as a record and auditability as a closure requirement.

12 Stage Transitions and Closure Lifecycle

The core calculus is finite-stage. It evaluates a closure formula at a specified review stage after typed atomic statuses and valid stopping designations have been supplied. Long-run learning, parameter drift, and institutional adaptation are therefore outside the basic formal system. For journal purposes the relevant dynamic requirement is narrower: a transition from stage $\mathbf{t}$ to stage $\mathbf{t}+1$ must be logged, justified, and tied to a review trigger.

Typical transition triggers include new evidence, audit completion, legal determination, ethical remedy, expiry, appeal, and conflict consolidation. Each transition must record the source of the change, the competent authority, dissenting evidence where present, and the appeal or review path. These rules integrate Expired_t , ValidRenewal_t , and CloseAction_{t+1} into the lifecycle of C . They also prevent silent status drift: a claim moves from C to T or F only through a recorded change in evidence, authority, remedy, review status, or expiry-controlled closure action.

13 Governance Interpretation

The intended interpretation of T-ACL is procedural. Closure is an institutional act: a responsible authority records that, for a specified purpose and stage, a claim may be treated as accepted, rejected, or still open. A claim marked T may be closed as accepted for that purpose. A claim marked F may be closed as rejected. A claim marked C must remain open, be remediated, or be escalated.

This interpretation separates three questions that are often conflated:

1. Is the underlying factual or normative issue true? 2. Is the institution currently permitted to close the claim? 3. If closure is not permitted, what must be reviewed, remedied, or escalated?

T-ACL answers only the second question directly. The first is answered by upstream evidence, legal, ethical, or argumentation procedures. The third is recorded in Audit_t .

Typical C triggers include:

- evidential conflict;
- missing evidence;
- semantic drift;
- unresolved dependency;
- ethical objection;
- legal uncertainty;
- audit failure;
- public contestation;
- subgroup harm;
- parameter sensitivity.

These triggers are not formally identical. T-ACL compresses them into the same procedural status only at the level of closure, while preserving diagnostic differences through Reason_t and Audit_t . Future versions may refine the audit-level reason-code taxonomy, but the present calculus keeps the semantic status set fixed at $V = \{T, F, C\}$.

A decision procedure using T-ACL should include stopping rules:

- maximum C lifetime, with no silent renewal;
- mandatory escalation, suspension, or closure action when C expires;
- immediate F when a valid unrevisable blocking condition is established;
- suspension if audit completeness, role separation, or parameter stability fails;
- appeal rights for affected parties.

These stopping rules prevent C from becoming a tool of indefinite delay. They also answer a common objection: C is not a permission to avoid decision. It is an obligation to keep a claim open only under public reasons and only within a governed review structure.

14 Documentary Evaluation Across Three Public-Record Cases

Access to internal audit records, legal deliberations, model-risk committee minutes, and institutional closure decisions is often restricted. The evaluation is therefore documentary and construct-validity oriented. It does not claim to be a field experiment, causal evaluation, source-code audit, or proof that T-ACL improves institutional outcomes. It asks a narrower question: can the calculus reconstruct closure-relevant distinctions that binary approval systems tend to collapse?

The case set uses maximum variation rather than statistical sampling. EEOC v. iTutorGroup concerns private employment screening and protected-age exclusion. Michigan’s MiDAS controversy concerns automated public-benefit fraud adjudication, notice, appeal, and property interests. The Dutch SyRI judgment concerns public-sector risk profiling, privacy, transparency, and judicial invalidation. Together they test whether the same closure interface can represent documentary insufficiency, alleged rule-level exclusion, due-process failure, rights-based legal blocking, invalid or contested stopping authority, audit incompleteness, expiry-controlled delay, and review of a corrected system.

14.1 EEOC v. iTutorGroup: Protected-Age Exclusion in Online Hiring

According to EEOC public materials, the agency alleged that iTutorGroup’s online recruitment software automatically rejected female applicants aged 55 or older and male applicants aged 60 or older, and that more than 200 qualified applicants were rejected because of age. The case later settled for \$365,000 plus non-monetary relief, including training, anti-discrimination policy, injunctions against discriminatory hiring based on age or sex and against requesting birth dates, and EEOC monitoring if U.S. tutor hiring resumed (U.S. Equal Employment Opportunity Commission 2022; 2023). These records do not provide source code, applicant-level data, or an independent technical audit. They do provide enough material to test whether T-ACL can distinguish public-record insufficiency, legal contestation, valid blocking failure for the same protected-age rule, and a fresh review path for any corrected future system.

Let G_IT mean that the online tutor recruitment system is closure-eligible for U.S.-based tutor hiring. Use P for performance or job-related selection sufficiency, B for absence of a protected-age exclusion rule, E for subgroup equity and remediation status, L for legal compliance, A for auditability and contestability, R for remediation and monitoring duties, and H for validated hard-stop authority. The closure formula is:

$$G_IT = (((((P \wedge_d B) \wedge_d E) \wedge_d L) \wedge_d A) \wedge_d R).$$

H is not an ordinary deployment dependency in the formula. It is an admissibility condition for stopping authority: $\text{ValidStop}_t(L)$ or $\text{ValidStop}_t(B)$ is admissible only if $H = T$.

The coding basis is deliberately public-record limited:

Atom	Public-record basis	Coding rule	Alternative coding checked
P	public record does not disclose source code, validation data, or job-related performance evidence	C unless performance evidence is independently available	treating P as T would import non-public assumptions
B	EEOC alleged an age-threshold rejection rule; settlement enjoined discriminatory hiring and birth-date requests	F for continued use of the same protected-age rule after the allegation is legally resolved; C while the rule is only alleged	B separates rule-level exclusion from general equity impact
E	alleged disparate rejection of older applicants and settlement relief	C while remediation and monitoring duties remain open	coding E as final F would erase the possibility of a corrected future system
L	EEOC enforcement action, settlement, injunctions, and monitoring terms	F plus eligible stop only after competent legal authority is validated	before validation, legal contestation remains C
A	public record lacks internal audit trail, applicant-level data, source code, and independent technical audit	C unless audit completeness and contestability fields are available	A prevents public-record silence from becoming approval
R	settlement required training, policy change, injunctions, and possible EEOC monitoring	C while remedial obligations remain open	R distinguishes rejection of the old rule from review of a corrected system
H	legal authority, scope, appeal path, and role separation for a hard stop	T only when stopping authority is competent and role-separated	proposed stop with unresolved authority is coded C

The staged coding is:

- $\mathbf{t0}$, public-record baseline reconstructed from later materials: $P = C$, $B = C$, $E = C$, $L = C$, $A = C$, $R = C$; no valid stop and H not yet established; output C.
- $\mathbf{t1}$, EEOC lawsuit filed and protected-age rejection publicly alleged: $P = C$, $B = C$, $E = C$, $L = C$, $A = C$, $R = C$; proposed stop only and $H = C$; output C.
- $\mathbf{t2}$, settlement/decreed period for continued use of the same protected-age rule: $P = C$, $B = F$, $E = C$, $L = F$, $A = C$, $R = C$; $H = T$ and valid stop on L or B; output (F, true) .
- $\mathbf{t3}$, hypothetical corrected future system after removing the protected-age rule: $P = C$, $B = T$, $E = C$, $L = C$, $A = C$, $R = C$; old stop is not inherited automatically; output C.

The iTutorGroup reconstruction controls several modelling hazards. $\mathbf{t0}$ is based on the public-document horizon rather than presumed internal corporate belief. B, E, and L separate rule-level protected-age exclusion, subgroup equity/remediation status, and legal closure. A remains C at $\mathbf{t2}$ because public records still lack the internal audit trail and applicant-level materials. The overall output is nevertheless (F, true) because validated legal or rule-level blocking on B or L propagates through the deliberative closure formula.

14.2 Michigan MiDAS: Automated Fraud Adjudication and Due Process

Michigan’s MiDAS controversy tests a different institutional setting: automated public-benefit fraud adjudication. Public litigation and official settlement materials describe challenges to the state’s automated unemployment-fraud determinations, including allegations that claimants were accused, assessed, or deprived of property without adequate notice and contestability. Michigan’s Attorney General announced a \$20 million settlement in the civil-rights class action, while state-court and federal-court records treated the controversy as a serious due-process and automated-adjudication problem rather than a simple accuracy dispute (Cahoo v. SAS Analytics Inc. 2019; Bauserman v. Unemployment Insurance Agency 2022; Michigan Department of Attorney General 2022).

Let G_{MI} mean that an automated unemployment-fraud adjudication workflow is closure-eligible for deployment or continued use. Use N for notice and appeal adequacy, Q for determination reliability and error-control evidence, L for legal and due-process compliance, A for auditability and contestability, R for remediation or restitution obligations, and H for validated hard-stop authority. The closure formula is:

$$G_{MI}I = (((N \wedge_d Q) \wedge_d L) \wedge_d A) \wedge_d R).$$

The staged reconstruction is:

- $m0$, public operation before adequate public challenge record is available: $N=C$, $Q=C$, $L=C$, $A=C$, $R=C$; $H = C$; output C . The public record is insufficient for approval because the workflow’s notice, appeal, and reliability conditions are not closure-ready.
- $m1$, audit/litigation stage: $N=C$, $Q=C$, $L=C$, $A=C$, $R=C$; proposed stop only; output C . Litigation or audit allegations alone do not make the modeler invent a hard stop; they force corrective non-closure until authority and scope are settled.
- $m2$, validated due-process failure for the same adjudication pathway: $N=F$, $Q=C$, $L=F$, $A=C$, $R=C$. If $H = T$, a valid stop on N or L produces (F, true) . If authority or role separation is contested, $H = C$ and the output remains C rather than T .
- $m3$, redesigned workflow with human review, notice, appeal, and remediation obligations still open: $N=C$, $Q=C$, $L=C$, $A=C$, $R=C$; the prior stop is not inherited automatically by a materially changed process; output C until the new review conditions close.

MiDAS adds two checks that iTutorGroup does not supply. First, the relevant harm is not only classification bias; it concerns administrative adjudication, notice, appeal, and property deprivation. Second, a high-stakes public workflow may deserve C even when no single technical metric has yet failed, because closure depends on contestability and due-process conditions. T-ACL represents this as a closure problem: missing procedural safeguards do not become acceptance by default, but allegations also do not become a valid stop unless competent authority and role separation are satisfied.

14.3 Dutch SyRI: Risk Profiling, Privacy, and Judicial Blocking

The Dutch SyRI case tests a stronger legal-blocking scenario. SyRI, the Systeem Risico Indicatie, was a statutory risk-indication system used for detecting welfare and tax fraud risks. In 2020 the District Court of The Hague held that the SyRI legislation, as applied in the challenged framework, violated Article 8 of the European Convention on Human Rights because it failed to provide sufficient safeguards, transparency, and proportionality for interference with private life (District Court of The Hague 2020). Unlike a mere audit concern, the judgment supplies a public legal basis for blocking continued use of the challenged risk-indication framework.

Let G_{SY} mean that the SyRI risk-indication framework is closure-eligible for public-sector fraud-risk deployment. Use P for predictive or targeting sufficiency, T for transparency and explainability, R for rights and proportionality, L for legal compliance, A for contestability and auditability, and H for validated judicial or legal stopping authority. The closure formula is:

$$G_{SY}Y = (((P \wedge_d T) \wedge_d R) \wedge_d L) \wedge_d A).$$

The staged reconstruction is:

- $s0$, statutory deployment before judicial invalidation: $P=C$, $T=C$, $R=C$, $L=C$, $A=C$; $H = C$; output C . The public record does not justify closure as acceptance because transparency, rights, proportionality, and contestability conditions remain unsettled.
- $s1$, civil-society challenge and litigation: $P=C$, $T=C$, $R=C$, $L=C$, $A=C$; proposed stop only; output C . Contestation is not yet a hard stop, but it prevents acceptance.
- $s2$, District Court judgment: $P=C$, $T=C$, $R=F$, $L=F$, $A=C$; $H = T$; valid stop on R or L ; output (F, true) .
- $s3$, hypothetical redesigned risk-assessment system under a new legal and technical architecture: $P=C$, $T=C$, $R=C$, $L=C$, $A=C$; old stop is not inherited automatically; output C until transparency, rights, legal, and contestability conditions close.

SyRI supplies the cleanest example of a rights-based stop. T-ACL does not need to decide whether the predictive purpose was administratively useful. Once a competent legal authority establishes an unresolvable rights or legality failure for the challenged framework, unresolved performance or administrative-utility claims cannot keep the deployment claim alive. The result is blocking F for the old framework and C for a materially redesigned future one.

14.4 Cross-Case Comparison

The three reconstructions strengthen different parts of the calculus. iTutorGroup tests whether rule-level protected-class exclusion, unresolved auditability, and corrected-system review can be separated in employment screening. MiDAS tests whether automated public-benefit adjudication can remain C when notice, appeal, audit, and remediation are incomplete, while still becoming blocking F when competent authority validates a due-process stop for the same pathway. SyRI tests whether a judicial rights-based stop overrides unresolved performance claims in public risk profiling.

The comparison also narrows the empirical claim. T-ACL is not validated here as a superior governance intervention. The cases instead support representational adequacy: a single closure-status interface can encode three real public-record trajectories without forcing every negative record into final rejection, every incomplete record into approval, or every legal stop into discretionary delay. That is the level at which the present article makes an empirical contribution. Outcome validity would require institutional records, affected-party evidence, independent technical audit, and comparative deployment study.

15 Related Work and Functional Positioning

T-ACL sits between formal non-binary evaluation and practical AI assurance. K3 and related three-valued logics made non-binary evaluation familiar (Kleene 1952; Lukasiewicz 1920). Priest’s LP and other paraconsistent logics show how conflict can be handled without triviality (Priest 1979; da Costa 1974). Belnap’s four-valued logic and Dunn-style information semantics distinguish informational states such as both and neither (Belnap 1977; Dunn 1976). AGM belief revision and dynamic epistemic logic model changes in epistemic state (Alchourrón, Gärdenfors, and Makinson 1985; Gärdenfors 1988; van Benthem 2011). Dung-style argumentation frameworks model attack, defence, and undecided status among arguments (Dung 1995).

These traditions supply much of the upstream machinery that a governance review may need. T-ACL occupies a different layer: it asks how such outputs affect the institutional act of closure. An argumentation system may leave an equity claim undecided; a legal system may establish a binding prohibition; a provenance system may show that a required source trace is missing; an assurance case may contain an unresolved defeater. T-ACL maps those outputs into a closure status that distinguishes acceptance, rejection, corrective non-closure, and validated blocking failure.

Legal and institutional frameworks sharpen the same point. Deontic logic, input/output logic, de-feasible reasoning, and formal legal reasoning model obligations, permissions, exceptions, priorities, and legal conflict. Workflow and process-verification frameworks model tasks, traces, transitions, and completion states. Provenance and accountability logics record source histories. Assurance cases and Goal Structuring Notation organize claims, evidence, assumptions, and defeaters in safety and assurance arguments (Kelly and Weaver 2004). T-ACL can consume outputs from each of these frameworks while keeping the closure question explicit.

Recent AI governance work makes this interface practically salient. Algorithmic auditing research emphasizes that audits require institutional process, evidence access, and contestability, not only metrics (Kroll et al. 2017; Raji et al. 2020; Lam et al. 2024). Studies of Local Law 144 show how a legally required audit can still leave open questions about scope, disclosure, and practical accountability (Groves et al. 2024). The EU Artificial Intelligence Act establishes risk-based obligations and conformity-assessment routes for AI systems (European Parliament and Council 2024). NIST’s AI RMF and Generative AI Profile emphasize governance, mapping, measurement, management, documentation, and evaluation of risks (NIST 2023; NIST 2024). These regimes require decisions about whether assurance packages are complete enough for action. T-ACL supplies a compact closure interface for that decision.

This positioning also fixes the contribution boundary. T-ACL converts informational or argumentative incompleteness into auditable closure status; it records when update or undecidedness keeps a claim in C; it passes non-waivable legal, safety, or rights failures into ValidStop_t and Block_t ; and it translates missing workflow traces, provenance gaps, or assurance-case defeaters into closure consequences. The calculus is deliberately coarser than Belnap-style information semantics, thinner than argumentation theory, and less domain-specific than legal reasoning. Its theoretical value lies in closure-control separation: the same negative finding can be represented as remediable C, ordinary F, or blocking F depending on authority, reason, expiry, and audit conditions.

16 Limitations

The most important limitation is granularity. C groups evidence insufficiency, evidence conflict, semantic drift, ethical contestation, and audit failure under one closure status. That compression is useful only if the diagnostic reason remains available in Audit_t . The reason-code vocabulary used here is a minimum working taxonomy; domain-specific implementations may extend it.

The second boundary concerns evidence generation. α_t and Θ_t are interfaces. T-ACL audits the form, authority link, and contestability of α_t ; substantive validation of the upstream status assignment requires independent coding, expert review, or access to institutional audit records.

The third boundary is temporal. Conflict consolidation and C-expiry rules make the calculus non-monotonic across stages. The fixed-stage results in this article show deterministic evaluation once a stage is specified. A fuller dynamic theory would need to model long-run institutional learning, strategic delay, and adversarial manipulation of review cycles.

The empirical boundary is also substantial. The evaluation is a public-record reconstruction and representational adequacy test. Public records can test whether the notation reconstructs closure-relevant distinctions in a real controversy; field experiments, independent technical audits, affected-party evidence, and comparative deployment studies would be needed to evaluate governance outcomes.

Finally, ValidStop_t rests on institutional authority. Public stopping designations, role separation, expiry metadata, and appeal paths are preconditions for a valid closure model. Legal, professional, and political processes determine who may set thresholds, define evidence functions, or impose non-waivable stops. A deployable governance system would need domain-specific evidence standards, public appeal procedures, independent audit, and safeguards against captured review authority.

17 Conclusion

AI assurance workflows often accumulate evidence before they produce a defensible closure decision. T-ACL addresses that downstream point. It introduces C as a procedural corrective status and separates factual conjunction from deliberative conjunction, so that ordinary factual reasoning remains intact while governance review can represent suspended closure.

The core contribution is the closure-control separation. Remediable non-closure, closure-level rejection, and non-waivable blocking failure receive different formal treatments. A claim can remain C because review is still owed; it can become F because rejection is now procedurally available; or it can become blocking F because a validated legal, safety, or rights constraint has been triggered. This separation is small, but it matters in AI governance settings where binary approval can hide unfinished review or where indefinite deferral can hide responsibility.

T-ACL is best read as a component in a larger assurance architecture. Empirical validation, legal authority, and institutional judgement remain upstream and surrounding conditions; the calculus supplies the closure-status interface that connects them to a reproducible review outcome. The next step is practical: implement the reasoner, code additional public cases independently, and compare the resulting closure records with ordinary workflow and assurance-case representations.

References

- Alchourrón, C. E., Gärdenfors, P., and Makinson, D. (1985). On the logic of theory change: partial meet contraction and revision functions. *Journal of Symbolic Logic*, 50(2), 510-530.
- Amnesty International. (2021). *Xenophobic machines: Discrimination through unregulated use of algorithms in the Dutch childcare benefits scandal*. Amnesty International.
- Barocas, S. and Selbst, A. D. (2016). Big data’s disparate impact. *California Law Review*, 104, 671-732.
- Bauserman v. Unemployment Insurance Agency, 509 Mich. 673; 983 N.W.2d 855 (Mich. 2022).
- Belnap, N. D. (1977). A useful four-valued logic. In J. M. Dunn and G. Epstein (eds.), *Modern Uses of Multiple-Valued Logic*. Reidel.
- Bellamy, R. K. E. et al. (2019). AI Fairness 360: An extensible toolkit for detecting and mitigating algorithmic bias. *IBM Journal of Research and Development*, 63(4/5), 4:1-4:15.
- Cahoo v. SAS Analytics Inc., 912 F.3d 887 (6th Cir. 2019).
- da Costa, N. C. A. (1974). On the theory of inconsistent formal systems. *Notre Dame Journal of Formal Logic*, 15(4), 497-510.
- District Court of The Hague. (2020). SyRI legislation in breach of European Convention on Human Rights. February 5, 2020. <https://www.rechtspraak.nl/organisatie-en-contact/organisatie/rechtbanken/rechtbank-den-haag/nieuws/2020/02/syri-legislation-in-breach-of-european-convention-on-human-rights>.
- Dung, P. M. (1995). On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games. *Artificial Intelligence*, 77(2), 321-357.
- Dunn, J. M. (1976). Intuitive semantics for first-degree entailments and coupled trees. *Philosophical Studies*, 29, 149-168.
- Equal Employment Opportunity Commission. (n.d.). Age Discrimination in Employment Act of 1967. <https://www.eeoc.gov/statutes/age-discrimination-employment-act-1967>.
- Equal Employment Opportunity Commission. (2022). EEOC sues iTutorGroup for age discrimination. May 5, 2022. <https://www.eeoc.gov/newsroom/eeoc-sues-ityorgroup-age-discrimination>.
- Equal Employment Opportunity Commission. (2023). iTutorGroup to pay \$365,000 to settle EEOC discriminatory hiring suit. September 11, 2023. <https://www.eeoc.gov/newsroom/ityorgroup-pay-365000-settle-eeoc-discriminatory-hiring-suit>.
- European Parliament and Council. (2024). Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). *Official Journal of the European Union*, 12 July 2024.
- Geburu, T., Morgenstern, J., Vecchione, B., Vaughan, J. W., Wallach, H., Daume III, H., and Crawford, K. (2021). Datasheets for datasets. *Communications of the ACM*, 64(12), 86-92.
- Google PAIR. (n.d.). *People + AI Guidebook*. Google People + AI Research.
- ISO/IEC. (2023a). *ISO/IEC 42001:2023 Artificial intelligence – Management system*. International Organization for Standardization.
- ISO/IEC. (2023b). *ISO/IEC 23894:2023 Artificial intelligence – Guidance on risk management*. International Organization for Standardization.
- Gärdenfors, P. (1988). *Knowledge in Flux: Modeling the Dynamics of Epistemic States*. MIT Press.
- Green, T. J., Karvounarakis, G., and Tannen, V. (2007). Provenance semirings. In *Proceedings of PODS*.
- Horty, J. F. (2012). *Reasons as Defaults*. Oxford University Press.
- Kleene, S. C. (1952). *Introduction to Metamathematics*. North-Holland.
- Lukasiewicz, J. (1920). On three-valued logic. Reprinted in L. Borkowski (ed.), *Selected Works*. North-Holland.
- Makinson, D. and van der Torre, L. (2000). Input/output logics. *Journal of Philosophical Logic*, 29, 383-408.
- NIST. (2023). *Artificial Intelligence Risk Management Framework (AI RMF 1.0)*. National Institute of Standards and Technology.
- NIST. (2024). *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1)*. National Institute of Standards and Technology.
- National Telecommunications and Information Administration. (2024). *Artificial Intelligence Accountability Policy Report*. U.S. Department of Commerce.
- Office of Management and Budget. (2024). *Memorandum M-24-10: Advancing Governance, Innovation, and Risk Management for Agency Use of Artificial Intelligence*. Executive Office of the President.
- Treasury Board of Canada Secretariat. (2024). *Algorithmic Impact Assessment tool*. Government of Canada.

- Prakken, H. and Sartor, G. (1996). A dialectical model of assessing conflicting arguments in legal reasoning. *Artificial Intelligence*, 77(2), 361-397.
- Priest, G. (1979). The logic of paradox. *Journal of Philosophical Logic*, 8(1), 219-241.
- Rawls, J. (1993). *Political Liberalism*. Columbia University Press.
- Reiter, R. (1980). A logic for default reasoning. *Artificial Intelligence*, 13(1-2), 81-132.
- Rescher, N. (1969). *Many-Valued Logic*. McGraw-Hill.
- van Benthem, J. (2011). *Logical Dynamics of Information and Interaction*. Cambridge University Press.
- Wiener, N. (1948). *Cybernetics: Or Control and Communication in the Animal and the Machine*. MIT Press.
- Floridi, L. (2019). Translating principles into practices of digital ethics: five risks of being unethical. *Philosophy and Technology*, 32, 185-193.
- Kelly, T. and Weaver, R. (2004). The Goal Structuring Notation: a safety argument notation. *Proceedings of the Dependable Systems and Networks Workshop on Assurance Cases*.
- Kroll, J. A. et al. (2017). Accountable algorithms. *University of Pennsylvania Law Review*, 165, 633-705.
- Lam, M. S. et al. (2024). A framework for assurance audits of algorithmic systems. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency*.
- Groves, L. et al. (2024). Auditing work: exploring the New York City algorithmic bias audit regime. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency*.
- Mitchell, M. et al. (2019). Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency*, 220-229.
- Michigan Department of Attorney General. (2022). State of Michigan announces settlement of civil rights class action alleging false accusations of unemployment fraud. October 20, 2022. <https://www.michigan.gov/ag/news/press-releases/2022/10/20/som-settlement-of-civil-rights-class-action-alleging-false-accusations-of-unemployment-fraud>.
- New York City Department of Consumer and Worker Protection. (2023). *Automated employment decision tools*. City of New York.
- Raji, I. D. et al. (2020). Closing the AI accountability gap: defining an end-to-end framework for internal algorithmic auditing. In *Proceedings of FAT*.
- Selbst, A. D., Boyd, D., Friedler, S. A., Venkatasubramanian, S., and Vertesi, J. (2019). Fairness and abstraction in sociotechnical systems. In *Proceedings of FAT*.
- van der Aalst, W. M. P. (1998). The application of Petri nets to workflow management. *Journal of Circuits, Systems, and Computers*, 8(1), 21-66.
- von Wright, G. H. (1951). Deontic logic. *Mind*, 60, 1-15.